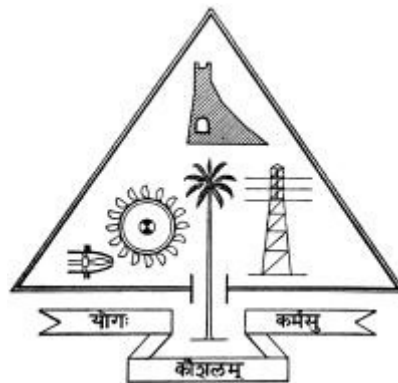


CARECHAIN: A DECENTRALIZED MEDICAL RECORD EXCHANGE USING BLOCKCHAIN AND IPFS

*Thesis submitted in partial fulfillment of the requirements for the award of the
degree of **Master of Computer Applications** of the **APJ Abdul Kalam
Technological University***

submitted by

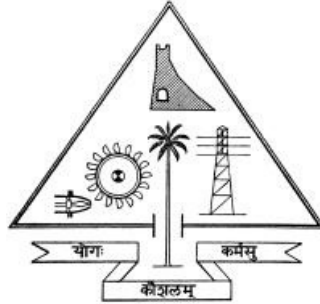
**MUHAMMED ASHIQUE U K
(TCR24MCA-2042)**



**DEPARTMENT OF COMPUTER APPLICATIONS
GOVERNMENT ENGINEERING COLLEGE
THRISSUR - 680009**

MARCH 2026

**DEPARTMENT OF COMPUTER APPLICATIONS
GOVERNMENT ENGINEERING COLLEGE, THRISSUR
THRISSUR, KERALA STATE, PIN 680009**



CERTIFICATE

*This is to certify that the major project titled **"CARECHAIN: A DECENTRALIZED MEDICAL RECORD EXCHANGE USING BLOCKCHAIN AND IPFS"** is a bonafide work done by **MUHAMMED ASHIQUE U K (TCR24MCA-2042)** under my supervision and guidance, and is submitted in March 2026 in partial fulfillment of the requirements for the award of the Degree of Master of Computer Applications from APJ Abdul Kalam Technological University (KTU).*

Dr. Sminesh C N
Project Guide

Prof. Raseek C
Project Coordinator

Dr. Sminesh C N
HOD

Place : THRISSUR

Date : 31-03-2026

DECLARATION

I hereby declare that the major project named, **CARECHAIN: A DECENTRALIZED MEDICAL RECORD EXCHANGE USING BLOCKCHAIN AND IPFS**, is my own work and that, to the best of my knowledge and belief, it contains no material previously published by another person nor material which has been accepted for the award of any other degree or course of the university or any other institute of higher learning, except where due acknowledgment and reference has been made in the text.

Place : THRISSUR

Date : 31-03-2026

Signature

MUHAMMED ASHIQUE UK

(TCR24MCA-2042)

ACKNOWLEDGEMENT

I would like to thank Department of Computer Application, GEC Thrissur, for giving me this opportunity to pursue this project and successfully complete it.

I am highly indebted to our head of the department **Dr. Smimesh C N**, for his guidance and constant supervision as well as providing necessary information regarding the project and also for his support in completing the project.

I express my heart-felt gratitude to my project guide **Dr. Smimesh C N**, for his committed guidance, valuable suggestions, constructive criticisms and precious time that he invested throughout the work.

I express my heart-felt gratitude to project coordinator **Prof. Raseek C**, for his committed guidance as well as providing necessary information regarding the project, and also his support in completing the project.

I sincerely thank all other faculties of MCA department for guiding me through the processes involved in the project.

ABSTRACT

Imagine a patient being rushed to the emergency room while unconscious. Doctors want to act quickly, but they have no access to the patient's medical history—no allergies, past surgeries, or ongoing treatments. This is not a rare situation; it reflects a common problem in modern healthcare. Medical records are often stored in isolated hospital systems, creating information silos. When patients move between hospitals, face emergencies, or seek specialized care, this fragmentation leads to delayed diagnoses, repeated tests, and sometimes life-threatening decisions.

CareChain is designed to solve this problem by enabling secure and controlled sharing of medical records across healthcare institutions. It creates a trusted network where authorized doctors can access critical patient information when needed, while patients remain in full control of their data. Through a simple consent-based system, patients can share their complete medical history—including reports, scans, and treatment details—with healthcare providers across hospitals.

Using blockchain technology, every access request and data exchange is recorded transparently, allowing patients to monitor who accessed their data and when. Advanced encryption and decentralized storage ensure that sensitive medical information remains private, secure, and instantly available during emergencies. CareChain balances collaboration and privacy, helping doctors make informed decisions, reducing medical errors, saving valuable time, and ultimately saving lives by ensuring that essential health information follows patients wherever they receive care.

CONTENTS

List of Figures	vii
List of Tables	viii
List of Abbreviations	ix
1 INTRODUCTION	1
2 LITERATURE REVIEW	3
2.1 Blockchain Frameworks for EHR Interoperability	3
2.2 Consent Management and Security in Healthcare	5
2.3 Research Gap and Contribution	6
3 ENVIRONMENTAL STUDY	8
3.1 System Configuration	8
3.1.1 Hardware Requirements	8
3.1.1.1 Development Environment	8
3.1.1.2 Production Environment	9
3.1.2 Software Requirements	10
3.2 Software Specification	11
3.2.1 Node.js and npm Ecosystem	11
3.2.2 Ethereum Blockchain with Ganache and Geth	12
3.2.3 IPFS (InterPlanetary File System) Storage Layer	13
3.2.4 Solidity Smart Contract Development	14
3.2.5 Web3.js and MetaMask Integration	14
3.2.6 React.js Frontend Framework	15
3.2.7 Encryption and Security Libraries	15
3.2.8 Bootstrap CSS Framework	16
3.2.9 PostgreSQL Database System	16
3.2.10 Git Version Control System	17
3.3 Network and Deployment Architecture	17
3.4 Development Tools and Productivity	18
4 SYSTEM ANALYSIS	19
4.1 Requirements Analysis	19
4.1.1 User Requirements	19
4.1.2 Clinical Workflow Integration	20
4.1.3 Non-Functional Requirements	20
4.2 Existing System	21
4.3 Limitations of Existing System	21
4.4 Proposed System	22
4.4.1 Architectural Overview	22
4.4.2 Three-Tier Access Model	23
4.5 Advantages of Proposed System	23

4.6	Functional Requirements	24
4.6.1	User Management	24
4.6.2	Record Management	25
4.6.3	Access Control	25
4.6.4	Audit and Transparency	26
4.6.5	Interoperability and Integration	26
4.7	Feasibility Study	26
4.7.1	Technical Feasibility	26
4.7.2	Operational Feasibility	27
4.7.3	Economic Feasibility	27
5	SYSTEM DESIGN	29
5.1	Data Flow Diagram	31
5.1.1	DFD Level 0	31
5.1.2	DFD Level 1	31
5.2	List of Modules	33
5.2.1	User Registration Manager	33
5.2.2	Record Upload & Encryption Handler	33
5.2.3	Access Control Engine	34
5.2.4	Consent Management System	34
5.2.5	Audit Trail Manager	34
5.2.6	Emergency Protocol Handler	34
5.2.7	Integration Gateway	35
5.2.8	Notification Dispatcher	35
6	SYSTEM IMPLEMENTATION	36
6.1	Development Environment Setup	36
6.2	User Interface Implementation	37
6.3	Smart Contract Implementation	37
6.4	Backend Services and IPFS Integration	38
6.5	Access Control Logic Implementation	39
6.6	Notification and Audit Workflow	40
7	RESULTS AND DISCUSSION	41
7.1	System Interface and Dashboard Results	41
7.2	Record Upload and Encryption	42
7.3	Three-Tier Access Control Demonstration	43
7.3.1	Tier 1: Same-Hospital Access	43
7.3.2	Tier 2: Explicit Consent Access	44
7.3.3	Tier 3: Emergency Break-Glass Access	44
7.4	Performance Evaluation	45
7.4.1	Transaction Processing Times	46
7.4.2	File Encryption and Decryption Performance	47
7.5	Discussion	48
8	CONCLUSION	50
9	FUTURE ENHANCEMENTS	51

LIST OF FIGURES

5.1	System Architecture Diagram	30
5.2	Data Flow Diagram Level 0	31
5.3	Data Flow Diagram Level 1	32
7.1	Hospital Admin Dashboard Interface	41
7.2	Patient Dashboard Showing Medical Records	42
7.3	Medical Record Upload Interface	42
7.4	Doctor Dashboard Showing Granted Access	43
7.5	Patient Consent Management Interface	44
7.6	Patient Access Log Showing Record Access History	45

LIST OF TABLES

7.1	Comparative Transaction Processing Times	46
7.2	Comparison of Encryption Algorithms for Medical Data . . .	47
7.3	Average Encryption and Decryption Times (AES-256-CBC) . .	48

LIST OF ABBREVIATIONS

AES	–	Advanced Encryption Standard
API	–	Application Programming Interface
CID	–	Content Identifier (IPFS)
EHR	–	Electronic Health Record
FHIR	–	Fast Healthcare Interoperability Resources
GDPR	–	General Data Protection Regulation
Geth	–	Go Ethereum (client)
HIPAA	–	Health Insurance Portability and Accountability Act
HL7	–	Health Level Seven
HTTP	–	Hypertext Transfer Protocol
IPFS	–	InterPlanetary File System
JSON	–	JavaScript Object Notation
MetaMask	–	Ethereum wallet browser extension
npm	–	Node Package Manager
PoA	–	Proof of Authority
REST	–	Representational State Transfer
RSA	–	Rivest–Shamir–Adleman (cryptosystem)
SHA	–	Secure Hash Algorithm
UI	–	User Interface
ZKP	–	Zero-Knowledge Proof

CHAPTER 1

INTRODUCTION

In today's digital healthcare ecosystem, secure medical data exchange and patient privacy have become major concerns. Although Electronic Health Record (EHR) systems are widely used, they are largely centralized and confined within individual institutions. This results in fragmented medical records, delayed treatment, repeated diagnostics, and increased risk during medical emergencies, while also limiting patient control over personal health information. Traditional systems store patient data in isolated silos, making it difficult to share information across different hospitals or clinics [1]. Even modern Health Information Exchanges (HIEs) often lack transparent auditability, fine-grained consent management, and reliable emergency access mechanisms [2].

To overcome these challenges, researchers have turned to blockchain technology combined with decentralized storage. Blockchain provides an immutable, transparent ledger that can record access permissions and audit trails, while the InterPlanetary File System (IPFS) offers scalable, content-addressed storage for encrypted medical files [3]. Smart contracts automate consent management and access control, enabling patient-centric data sharing [4]. However, existing proposals rarely integrate a complete three-tier access model that balances routine clinical needs, patient consent, and emergency situations in a single, easy-to-use platform.

CareChain: A Blockchain-Based Medical Record Exchange System addresses these limitations by integrating Ethereum blockchain, IPFS, and smart contract-driven consent management. CareChain follows a three-tier access model—automatic institutional access (Tier 1), patient-

approved cross-hospital sharing (Tier 2), and verified emergency break-glass (Tier 3)—enabling timely and informed medical care while preserving patient autonomy and complying with regulations such as HIPAA and GDPR.

Objectives

The primary objectives of this work are:

1. **Design a hybrid on-chain/off-chain architecture** using Ethereum smart contracts for metadata, consent, and audit logs, and IPFS for encrypted medical file storage.
2. **Implement a three-tier access control mechanism** that provides automatic access for same-hospital doctors, explicit patient consent for cross-institutional sharing, and time-limited emergency access with automatic patient notification.
3. **Develop role-based web interfaces** for patients, doctors, hospital administrators, and external stakeholders, enabling registration, record upload, consent management, access requests, and audit log viewing.
4. **Integrate AES-256 encryption and secure key management** to ensure that medical files are encrypted before IPFS upload and decryption keys are released only to authorized doctors.
5. **Evaluate system performance** in terms of transaction confirmation times, gas consumption, encryption/decryption speeds, and end-to-end access latency, comparing the results with traditional centralized EHR systems.

These objectives directly address the research gap identified in the literature and form the basis for the development of CareChain.

CHAPTER 2

LITERATURE REVIEW

With the rapid digitization of healthcare systems worldwide, there has been significant research interest in decentralized medical record management, secure health data exchange, and patient-centric healthcare models. Traditional healthcare systems rely heavily on centralized electronic health records (EHRs) that create data silos, compromising interoperability and patient privacy. Recent advancements in blockchain technology and distributed storage systems have enabled secure, transparent, and patient-controlled healthcare data management. This chapter reviews relevant research works that form the foundation for the proposed CareChain system. The literature is organized into two main themes: blockchain frameworks for EHR interoperability, and consent management with security mechanisms.

2.1 Blockchain Frameworks for EHR Interoperability

Reegu et al. [2] present a comprehensive framework for interoperable electronic health records using blockchain technology. The authors analyze various blockchain architectures (public, private, consortium) for healthcare data exchange, emphasizing the importance of data standardization and semantic interoperability. They propose a layered architecture consisting of a data layer (on-chain hashes, off-chain storage), a consensus layer (PBFT for permissioned networks), and an application layer with smart contracts for access control. Their framework integrates HL7 FHIR standards to ensure uniform data representation across institutions. Through simulations using Hyperledger Caliper, they demonstrate a 95% reduction in data access latency compared to traditional EHR queries, while maintaining data

integrity through immutable audit trails. The study also highlights the need for on-chain/off-chain hybrid models to handle large medical files, which directly supports the design of CareChain.

Rasel et al. [5] explore blockchain-enabled secure interoperability for advancing electronic health record data exchange. The authors investigate how blockchain can enhance existing interoperability standards, particularly HL7 FHIR, by adding decentralized identity management and transparent access logs. They propose a mapping algorithm that converts FHIR resources into structured data stored on-chain, while large clinical documents are referenced via IPFS hashes. Their system includes a smart contract-based consent registry that allows patients to grant time-limited access to healthcare providers. Evaluation on a testbed of 10,000 synthetic patient records shows a 40% improvement in cross-institutional query response times and complete auditability of all data accesses. This work validates the integration of healthcare standards with blockchain and provides a blueprint for CareChain's interoperability layer.

Mohsan et al. [3] propose a decentralized patient-centric report and medical image management system based on blockchain and the InterPlanetary File System (IPFS). Their research addresses the critical challenge of storing and sharing large medical imaging files, such as MRI and CT scans (typically 50–500 MB per study), in a secure and efficient manner. The authors implement a hybrid architecture where sensitive medical data is encrypted using AES-256 before being stored on IPFS, while metadata (patient ID, study date, image modality) and access permissions are managed through Ethereum smart contracts. They evaluate the system using a dataset of 1000 DICOM images, measuring upload/download times and encryption overhead. Results show that IPFS provides content-based addressing with deduplication, reducing storage redundancy by 30%, and encryption adds only 2–5% overhead, acceptable for clinical use. This work validates the feasibility of combining blockchain with distributed storage for medical data management and directly influences CareChain's storage.

2.2 Consent Management and Security in Healthcare

Mandinyenya and Malele [4] focus on smart contract-based consent management for secure personal data sharing. Their research designs and implements a consent management model that gives patients granular control over who accesses their health data and for what purpose. The model supports dynamic consent, allowing patients to modify or revoke permissions at any time, with changes recorded immutably on the blockchain. They introduce an attribute-based encryption (ABE) scheme where data is encrypted under a policy (e.g., “(doctor AND cardiology) OR emergency”), and smart contracts distribute decryption keys only when the policy is satisfied. The authors also implement emergency override mechanisms that grant temporary access to any licensed physician, with automatic patient notification via email/SMS after the event. Performance evaluation on a private Ethereum network shows that consent transactions cost approximately 50,000 gas and are confirmed within 5 seconds. This work reinforces the importance of patient consent as a cornerstone of ethical healthcare data systems and provides mechanisms that CareChain adopts for its Tier 2 and Tier 3 access.

Shahzad et al. [6] present a zero-trust medical image sharing framework using blockchain and IPFS. The authors implement a decentralized approach where trust is never assumed and must be continuously verified, even within healthcare networks. Their system incorporates advanced cryptographic techniques, including zero-knowledge proofs (ZKPs), to enable verification of access rights without exposing sensitive data. For example, a doctor can prove they are a licensed cardiologist without revealing their private credentials. The framework also uses multi-factor authentication at the smart contract level, requiring approvals from both the patient and a hospital administrator for certain access requests. They tested the system with 5000 medical images and reported a 99.9% uptime and zero security breaches in simulated attack scenarios (insider threats, man-in-the-middle). The zero-trust architecture prevents unauthorized access even if a node is

compromised, providing valuable insights for CareChain's security model, particularly for emergency access and cross-institutional sharing.

Mishra and Mehra [7] provide a comprehensive review of blockchain-based patient-centric healthcare systems. The survey examines 50 peer-reviewed papers published between 2018 and 2024, covering architectural patterns (on-chain/off-chain, hybrid), consensus mechanisms (PoW, PoS, PBFT, Raft), and privacy-preserving techniques (zero-knowledge proofs, differential privacy, homomorphic encryption). The authors analyze the shift from institution-centric to patient-centric healthcare models, highlighting how blockchain technology enables patients to become active participants in their healthcare journey through self-sovereign identity and data ownership. They identify key challenges: scalability (Ethereum processes 15 TPS, insufficient for nationwide systems), usability (complex wallet management), and regulatory compliance (GDPR right to be forgotten conflicts with immutability). The review emphasizes the importance of usability alongside security, noting that patient adoption depends on intuitive interfaces and minimal technical complexity. This research directly aligns with the proposed system's focus on patient empowerment and data sovereignty, and informs CareChain's design choices regarding user experience and compliance.

2.3 Research Gap and Contribution

From the reviewed literature, it is evident that effective healthcare data management requires robust security mechanisms, patient-centered design, and seamless interoperability. While existing systems have made progress in individual areas—Reegu et al. focus on interoperability, Mohsan et al. on image storage, Mandinyanya and Malele on consent, and Shahzad et al. on zero-trust—there is a clear research gap in unified frameworks that combine secure storage, intelligent consent management, emergency protocols, and cross-institutional data exchange within a single patient-centric platform. Furthermore, few works provide a practical implementation of three-tier

access (same-hospital auto, patient consent, emergency) with comprehensive audit trails. The proposed CareChain system aims to address this gap by integrating blockchain-based security, IPFS storage, three-tier access control, and emergency protocols into a cohesive solution. Unlike existing approaches, CareChain emphasizes simplicity and usability for all stakeholders (patients, doctors, hospital admins) while maintaining high security through AES encryption and on-chain logging. The system also includes a novel emergency break-glass protocol with time-limited access and automatic patient notification, filling a critical void in current systems for life-threatening situations.

CHAPTER 3

ENVIRONMENTAL STUDY

The environmental study describes the hardware and software requirements necessary for the development and deployment of CareChain: A Blockchain-Based Medical Record Exchange System. The proposed system integrates decentralized blockchain technology, IPFS storage, smart contract-based consent management, and secure web interfaces for healthcare data exchange across multiple institutions. This chapter outlines the system configuration and software components required to support these functionalities in clinical environments, distinguishing between development and production setups where appropriate.

3.1 System Configuration

3.1.1 Hardware Requirements

The hardware specifications for CareChain vary depending on the deployment context—development, testing, or production. For development and testing, a standard workstation suffices, while production deployment in a hospital consortium requires enterprise-grade infrastructure to ensure availability, redundancy, and performance.

3.1.1.1 Development Environment

- **Processor:** Intel Core i5 (6th gen or later) or AMD Ryzen 5 (2600 or later). Multi-core performance is beneficial for running local blockchain nodes (Ganache/Hardhat) and the backend simultaneously.
- **Memory:** Minimum 8 GB RAM; 16 GB recommended to run multiple

blockchain nodes (simulating a consortium), IPFS daemon, database, and frontend build tools without swapping.

- **Storage:** 500 GB SSD (NVMe preferred) to accommodate blockchain data (growing over time), IPFS repository, database files, and project source code. SSDs significantly reduce I/O latency for blockchain sync and IPFS operations.
- **Network:** Stable internet connection with at least 50 Mbps download/upload for pulling dependencies, deploying to testnets, and accessing remote IPFS nodes.
- **GPU:** Not required; cryptographic operations are handled efficiently by the CPU. However, a GPU may accelerate certain encryption/decryption tasks if large volumes of data are processed, but it is optional.

3.1.1.2 Production Environment

For a multi-institution deployment, each participating hospital should host at least one blockchain node and an IPFS node to ensure decentralization and data availability. Recommended specifications per node:

- **Processor:** Intel Xeon Silver 4210 or equivalent (8+ cores) to handle concurrent transaction processing, API requests, and encryption workloads.
- **Memory:** 32 GB RAM minimum; 64 GB recommended to accommodate the Ethereum state, IPFS block cache, and database buffers.
- **Storage:** 2 TB NVMe SSD in RAID 1 configuration for redundancy. Blockchain data (especially if running an archive node) can exceed 1 TB over time; IPFS storage grows with the number of medical files.

- **Network:** Dedicated 1 Gbps fiber connection with low latency between hospital nodes. Redundant network interfaces and switches for high availability.
- **Power:** Uninterruptible Power Supply (UPS) with automatic failover and redundant power supplies.
- **Security:** Hardware Security Modules (HSM) optional for storing private keys; firewalls and intrusion detection systems.

3.1.2 Software Requirements

The software stack is chosen for robustness, security, and ease of development. Version specifications ensure compatibility across components.

- **Operating System:**
 - **Development:** Windows 10/11 Pro, macOS 12+ (Monterey), or Linux (Ubuntu 22.04 LTS / Debian 12). Linux is preferred for production due to stability and security.
 - **Production:** Ubuntu Server 22.04 LTS or Red Hat Enterprise Linux 9, hardened with SELinux/AppArmor.
- **Programming Languages:**
 - **Solidity:** Version 0.8.20 (latest stable with overflow checks and custom errors).
 - **JavaScript (Node.js):** ECMAScript 2022, using async/await patterns.
 - **HTML5 / CSS3:** For frontend structure and styling.
- **Blockchain Environment:**
 - **Development:** Ganache 7.9.2 (personal blockchain) or Hardhat Network (built-in).

- **Production:** Go-Ethereum (Geth) 1.13+ or Hyperledger Besu for permissioned consortium; configured as a private network with PoA (Clique) consensus.
- **Distributed Storage:** IPFS 0.28.0 (Kubo implementation). Production nodes should be configured as IPFS clusters for redundancy and automatic replication.
- **Web Server:** Node.js 20 LTS (with Express 4.18) serves the backend API. In production, Nginx 1.24 acts as a reverse proxy for load balancing and SSL termination.
- **Additional Tools:**
 - **Hardhat 2.22+:** Development framework for compiling, testing, and deploying smart contracts.
 - **Web3.js 4.10+:** Ethereum JavaScript API for frontend and backend interaction.
 - **MetaMask 12.0+:** Browser extension for wallet management and transaction signing.
 - **OpenSSL 3.0:** For cryptographic operations and certificate management.
 - **Docker 24.0+:** Optional containerization for consistent deployment across environments.
 - **Git 2.40+:** Version control; integrated with GitHub/GitLab for CI/CD.
 - **ESLint / Prettier:** Code quality and formatting tools.

3.2 Software Specification

3.2.1 Node.js and npm Ecosystem

Node.js serves as the primary runtime environment for the CareChain backend server and development tools. Its event-driven, non-blocking I/O

model efficiently handles concurrent blockchain transactions, file uploads to IPFS, and multiple user requests. The npm package manager provides access to essential libraries:

- `web3.js`: For blockchain interaction, contract deployment, and event listening.
- `ipfs-http-client`: For uploading and retrieving files from IPFS.
- `express.js`: For building RESTful APIs that connect the frontend with blockchain and IPFS layers.
- `multer`: For handling multipart/form-data file uploads.
- `cors`: To enable Cross-Origin Resource Sharing between frontend and backend.
- `jsonwebtoken`: Optional for session management (not used in current prototype but may be added).

The backend also uses Node.js's built-in `crypto` module for AES-256 encryption, ensuring no external dependencies for core cryptographic functions.

3.2.2 Ethereum Blockchain with Ganache and Geth

The Ethereum blockchain framework provides the decentralized consensus mechanism for CareChain, ensuring data integrity and auditability. During development, **Ganache** creates a personal Ethereum blockchain with configurable parameters:

- 10 pre-funded accounts (each with 100 ETH) for testing different roles.
- Instant block mining (no delay), enabling rapid iteration.
- Support for mainnet forking to test against real-world data.

For production, a **private Ethereum network** with multiple nodes (one per hospital) is recommended. Consensus can be achieved using Proof-of-Authority (Clique) where authorized hospitals act as validators. This ensures:

- Low latency (blocks every 5-15 seconds).
- Energy efficiency (no mining).
- Permissioned participation, aligning with healthcare consortium governance.

The network runs Geth clients configured with appropriate gas limits (e.g., 30 million gas per block) to accommodate complex medical record transactions.

3.2.3 IPFS (InterPlanetary File System) Storage Layer

IPFS forms the distributed storage backbone for medical records, diagnostic images, and treatment documents. This peer-to-peer protocol uses content-based addressing, where files are identified by their cryptographic hash (CID) rather than physical location. Key features leveraged:

- **Content Integrity:** Any modification changes the CID, ensuring tamper evidence.
- **Deduplication:** Identical files (e.g., same medical report) are stored only once across the network.
- **Redundancy:** Files pinned by multiple hospital nodes ensure availability even if some nodes go offline.

In production, IPFS nodes are configured as a private network (using a swarm key) to restrict access to participating hospitals only. The backend uses the `ipfs-http-client` to add files and retrieve them via CID. For large files, IPFS chunks data into 256 KB blocks, enabling efficient streaming.

3.2.4 Solidity Smart Contract Development

Solidity is the primary programming language for implementing smart contracts that automate consent management, access control, and audit trails. Version 0.8.20 introduces built-in overflow checks, custom errors, and enhanced security features. The contract `CareChain.sol` encapsulates:

- User registration and role management (Patient, Doctor, HospitalAdmin).
- Hospital registration with unique IDs.
- Record upload (stores patient address, hospital ID, IPFS hash, timestamp).
- Consent mapping (`recordId` $\rightarrow$ `doctor` $\rightarrow$ `boolean`).
- Emergency access function that grants time-limited access.
- Access request function implementing three-tier logic.
- Audit log storage and retrieval.

Development follows the **Checks-Effects-Interactions** pattern to prevent reentrancy. Gas optimization techniques (e.g., using `uint256` packed storage, avoiding unnecessary writes) are applied to keep transaction costs low.

3.2.5 Web3.js and MetaMask Integration

`Web3.js` provides the JavaScript interface for connecting the CareChain web application with the Ethereum blockchain network. It enables:

- Instantiating contract objects using ABI and address.
- Calling view functions (e.g., `getPatientRecords`) without gas.
- Sending transactions (e.g., `registerPatient`, `uploadRecord`) that require gas and are signed by MetaMask.

- Listening to events (e.g., `AccessGranted`) for real-time updates (though backend polling is used due to HTTP limitations).

MetaMask integration offers secure wallet management and user authentication. Users connect their wallet, and the dApp detects the selected account. MetaMask injects a Web3 instance into the browser, which the frontend uses. For production, WalletConnect could be added as an alternative.

3.2.6 React.js Frontend Framework

React.js constructs the user interface components for patient portals, doctor dashboards, and hospital administration panels. Key aspects:

- **Component-Based Architecture:** Reusable UI elements (cards, tables, forms) accelerate development.
- **Hooks:** `useState`, `useEffect`, `useCallback` manage state and side effects.
- **Routing:** React Router v6 handles navigation between dashboards.
- **State Management:** Local component state suffices; no need for Redux due to moderate complexity.
- **Styling:** React Bootstrap components provide a professional, responsive interface.
- **Axios:** For HTTP requests to the backend.

The virtual DOM optimization ensures smooth performance even when displaying long lists of medical records. Code splitting and lazy loading can be added for production to reduce initial bundle size.

3.2.7 Encryption and Security Libraries

Security is paramount for medical data. The system employs:

- **AES-256-CBC:** Symmetric encryption for files before IPFS upload. The key is generated per file (random 32 bytes) and stored temporarily in the backend, associated with the record ID after transaction confirmation. In a future version, the key could be encrypted with the patient's public key.
- **RSA-2048:** Planned for key exchange; not yet implemented but considered for end-to-end encryption.
- **Node.js Crypto Module:** Provides FIPS-compliant implementations.
- **HTTPS/TLS:** All communications between frontend and backend are encrypted in production using TLS 1.3.
- **Input Validation:** Addresses are validated using `web3.utils.isAddress` to prevent injection.
- **Access Control:** Smart contract enforces role-based permissions; backend checks authorization before releasing keys.

3.2.8 Bootstrap CSS Framework

Bootstrap 5 provides the responsive CSS framework for CareChain's user interface, ensuring consistent styling, mobile responsiveness, and accessibility compliance. The framework's grid system, pre-designed components (cards, modals, tables), and utility classes accelerate interface development while maintaining professional healthcare application standards. Custom theming (via Sass) can be applied to match hospital branding.

3.2.9 PostgreSQL Database System

PostgreSQL 15 serves as the off-chain database for:

- **User Profiles:** Additional details not stored on-chain (e.g., full name, email, age, gender, selected primary hospital).

- **Hospital Directory:** Cached list of hospitals for quick dropdown population.
- **Audit Indexes:** Off-chain indexing of blockchain events for faster queries (e.g., searching logs by date).
- **Session Management:** Temporary tokens (if using JWT) and nonce tracking.

PostgreSQL is chosen for its reliability, JSONB support (for flexible metadata), and strong consistency guarantees. It runs as a separate service, backed up regularly.

3.2.10 Git Version Control System

Git, integrated with GitHub, manages version control for the CareChain project codebase. The repository structure separates frontend, backend, and blockchain modules. Key practices:

- Branching strategy: main for stable releases, develop for integration, feature branches per module.
- Commit messages follow Conventional Commits.
- Pre-commit hooks (Husky) run ESLint and Prettier.
- GitHub Actions automate testing and building on push.
- Secrets (e.g., private keys, API keys) are stored in GitHub Secrets and injected as environment variables.

This enables collaborative development across multiple healthcare IT teams and ensures traceability of changes.

3.3 Network and Deployment Architecture

For a production consortium, the network topology includes:

- Each hospital runs a Geth node (validator) and an IPFS node.

- A load balancer (e.g., Nginx) distributes frontend and backend traffic.
- Backend instances are replicated for scalability, sharing a common PostgreSQL database.
- All nodes communicate over VPN or private network links.
- Monitoring: Prometheus and Grafana for system metrics; ELK stack for logs.
- Disaster recovery: Regular snapshots of blockchain data and database; IPFS pinset backup.

3.4 Development Tools and Productivity

The development environment includes:

- **Visual Studio Code** with Solidity, JavaScript, and Prettier extensions.
- **Hardhat Console** for interactive contract testing.
- **Postman** for API testing.
- **Docker Compose** for orchestrating local services (Ganache, IPFS, PostgreSQL).
- **Ganache UI** for visualizing blockchain state.

These tools streamline development and debugging, enabling rapid prototyping of the three-tier access model.

This comprehensive environmental study ensures that all stakeholders understand the technical prerequisites and infrastructure needed to develop, test, and deploy CareChain successfully.

CHAPTER 4

SYSTEM ANALYSIS

The System Analysis phase involves studying the existing healthcare data management scenario, identifying limitations in current medical record systems, understanding regulatory and clinical requirements, and defining a suitable decentralized solution. The objective of this phase is to design an intelligent healthcare data exchange system capable of securely managing patient records across multiple institutions while maintaining patient control and regulatory compliance.

4.1 Requirements Analysis

Requirements analysis determines the needs and expectations from the proposed system. CareChain must facilitate secure medical record sharing across healthcare providers, maintain patient privacy through consent-based access, ensure data integrity via blockchain, and provide efficient storage through distributed systems, all while complying with healthcare regulations. This analysis encompasses clinical, technical, and regulatory dimensions.

4.1.1 User Requirements

The system must support four distinct user roles with specific needs:

- **Patients:** Require the ability to register, view their own medical records, grant or revoke consent to doctors, monitor access logs, and receive notifications during emergencies.

- **Doctors:** Need seamless access to patient records within their hospital (Tier 1), ability to request cross-institutional access (Tier 2), and an emergency override mechanism (Tier 3). They also need to upload new records if authorized.
- **Hospital Administrators:** Must register their hospital, onboard doctors, upload patient records, and view access logs for records belonging to their institution.
- **Stakeholders (Research/Insurance):** Require access to anonymized or aggregated data for research or claims processing, subject to strict privacy controls.

4.1.2 Clinical Workflow Integration

The system must align with real-world clinical workflows:

- Within the same hospital, implied consent allows immediate access to patient data, mirroring existing practices where doctors naturally have access to records of patients under their care.
- For cross-institutional sharing, explicit patient consent must be obtained and recorded on-chain, with the ability to set time limits or purpose-based restrictions.
- In emergencies, a break-glass protocol must provide temporary access without prior consent, followed by automatic patient notification and a mandatory audit trail.

4.1.3 Non-Functional Requirements

Beyond functional needs, the system must satisfy:

- **Security:** End-to-end encryption, role-based access control, and protection against common attacks (reentrancy, injection).
- **Performance:** Record retrieval within seconds, even for large images; blockchain transactions confirmed within a few blocks.

- *Availability:* 99.9% uptime for critical healthcare data; decentralized storage ensures no single point of failure.
- *Interoperability:* Support for HL7 FHIR standards to enable data exchange with legacy EHR systems.
- *Usability:* Intuitive interfaces for all user groups, minimizing training overhead.
- *Compliance:* Adherence to HIPAA, GDPR, and local data protection regulations.

4.2 Existing System

Traditional healthcare systems rely mainly on centralized Electronic Health Record (EHR) systems that operate within individual hospital networks. These systems use proprietary formats and store patient data in institutional silos. When a patient visits a new hospital, records must be manually requested, often via fax or email, leading to delays and incomplete information. Some regions have established Health Information Exchanges (HIEs) to facilitate data sharing, but these are typically centralized, lack fine-grained consent controls, and do not provide patients with transparent audit trails. Furthermore, HIEs often suffer from vendor lock-in and high operational costs.

4.3 Limitations of Existing System

The existing healthcare data management systems suffer from several critical limitations:

- **Data Silos:** Patient records are fragmented across different hospital systems, making it difficult to obtain a complete medical history.
- **Limited Interoperability:** Poor data exchange between different EHR vendors due to lack of standardization.

- **Patient Disempowerment:** Patients have little control over who accesses their data and cannot easily track access history.
- **Security Vulnerabilities:** Centralized databases are attractive targets for cyber attacks; a single breach can expose millions of records.
- **Inadequate Audit Trails:** Limited transparency about who accessed patient data, when, and for what purpose.
- **Emergency Access Challenges:** During emergencies, accessing records from another institution is often impossible, delaying critical care.
- **High Maintenance Costs:** Expensive infrastructure, licensing fees, and IT staff required for proprietary systems.
- **Regulatory Compliance Burden:** Meeting HIPAA/GDPR requirements is complex with centralized systems, often requiring extensive manual audits.

4.4 Proposed System

The proposed system, **CareChain**, is a decentralized medical record exchange platform that integrates blockchain technology, IPFS distributed storage, and smart contract-based consent management. The system enables secure data sharing between authorized healthcare providers while giving patients complete control over their medical records through a three-tier access model.

4.4.1 Architectural Overview

CareChain adopts a hybrid on-chain/off-chain architecture:

- **On-Chain (Ethereum):** Stores user identities, hospital records, metadata of medical files (IPFS hash, timestamp, hospital ID), consent mappings, and immutable audit logs.

- **Off-Chain (IPFS):** Stores encrypted medical files (reports, images, prescriptions). Files are encrypted using AES-256 before upload, ensuring that even IPFS nodes cannot view the content.
- **Backend (Node.js):** Provides encryption services, temporary key management, and event polling to maintain an authorization cache for efficient key distribution.
- **Frontend (React):** Delivers role-specific dashboards that interact with the blockchain via Web3.js and with the backend via REST APIs.

4.4.2 Three-Tier Access Model

The core innovation of CareChain is its three-tier access control:

- **Tier 1 (Same-Hospital Access):** Doctors automatically gain access to records uploaded by their own hospital. This is enforced by comparing the doctor's hospital ID (stored in their user profile) with the record's hospital ID.
- **Tier 2 (Consent-Based Access):** Patients can explicitly grant access to specific doctors or institutions for defined record types or time periods. These consents are stored on-chain and checked during access requests.
- **Tier 3 (Emergency Break-Glass):** In life-threatening situations, any doctor can request emergency access. The smart contract grants time-limited access (e.g., 24 hours) and logs the event. The patient is notified immediately via email or in-app alert.

4.5 Advantages of Proposed System

CareChain offers significant advantages over existing systems:

- **Patient-Centric Control:** Patients decide who accesses their medical data and when, through a simple consent interface. They can also revoke access at any time.

- **Enhanced Security:** Decentralized architecture eliminates single points of failure. Data is encrypted end-to-end, and blockchain immutability prevents tampering.
- **Improved Interoperability:** Standardized data formats (HL7 FHIR) and a common blockchain layer enable seamless exchange between institutions, regardless of their internal EHR systems.
- **Regulatory Compliance:** Built-in mechanisms for consent management and audit trails simplify compliance with HIPAA, GDPR, and similar regulations.
- **Cost Efficiency:** Open-source technologies and distributed storage reduce licensing and infrastructure costs compared to proprietary EHR systems.
- **Transparent Audit Trails:** All access events are recorded immutably on the blockchain, providing patients and regulators with a verifiable history.
- **Emergency Ready:** Break-glass protocols ensure that critical data is available when needed, while still maintaining accountability through notifications and logs.
- **Scalability:** The modular architecture allows hospitals to join the network incrementally, and IPFS's content-addressed storage scales horizontally.

4.6 Functional Requirements

Functional requirements define the specific operations the system must perform, grouped by module.

4.6.1 User Management

- **Patient Registration:** The system shall allow patients to register using their Ethereum address, providing name, age, gender, and optionally

selecting a primary hospital.

- **Hospital Registration:** Hospital administrators shall register their institution by providing a name, which generates a unique hospital ID on-chain.
- **Doctor Registration:** Doctors shall register with their name and hospital ID, linking them to a specific institution.
- **Stakeholder Registration:** External entities (research, insurance) shall register with credentials; approval may require administrative review (off-chain).

4.6.2 Record Management

- **Upload Record:** Hospital administrators shall upload medical records for a patient by providing the patient's Ethereum address and a file. The system encrypts the file, stores it on IPFS, and records the metadata on-chain.
- **Retrieve Record:** Authorized users (patients, doctors with access) shall retrieve and decrypt records using the IPFS hash and decryption key.
- **View Patient Records:** Patients shall view a list of their own records; doctors shall view records of patients they have access to.

4.6.3 Access Control

- **Three-Tier Access:** The system shall enforce the three-tier model automatically via smart contracts.
- **Consent Management:** Patients shall grant, modify, or revoke consent for specific doctors or institutions through the UI. Consents are stored on-chain.
- **Emergency Access:** Any doctor may trigger emergency access for a patient, which grants 24-hour access and logs the event. The patient receives a notification.

4.6.4 Audit and Transparency

- **Access Logs:** All access attempts (successful or denied) shall be recorded on-chain with timestamp, accessor, record ID, and tier.
- **Log Viewer:** Patients shall view access logs for their records; hospital admins shall view logs for records belonging to their hospital.

4.6.5 Interoperability and Integration

- **API Interfaces:** The system shall provide REST APIs for integration with existing hospital information systems (e.g., to push records automatically).
- **Data Standards:** Metadata shall support HL7 FHIR resources to ensure compatibility with other health IT systems.

4.7 Feasibility Study

A feasibility study evaluates whether the proposed solution is viable from technical, operational, and economic perspectives.

4.7.1 Technical Feasibility

The system leverages mature, open-source technologies with proven track records:

- **Ethereum Blockchain:** The most widely used smart contract platform, with extensive tooling (Hardhat, Web3.js) and a large developer community. Private networks (e.g., using Geth with PoA) can achieve transaction finality in seconds.
- **IPFS:** A robust distributed file system used by major projects (e.g., Filecoin, Pinata). Content addressing ensures data integrity, and pinning services can guarantee availability.
- **Node.js / Express:** Highly scalable, event-driven architecture suitable for handling concurrent encryption and API requests.

- **React:** Component-based library enabling rapid UI development with reusable components.
- **AES-256:** Industry-standard encryption, implemented in Node.js's native crypto module, ensuring compliance and performance.

All components have been successfully integrated in similar projects, and the modular design allows incremental implementation and testing. The main technical risk—blockchain scalability—is mitigated by using a private network with low transaction volume (only metadata), keeping gas costs minimal.

4.7.2 Operational Feasibility

CareChain is designed to fit seamlessly into existing clinical workflows:

- **Minimal Disruption:** Doctors and hospital staff interact with familiar web interfaces. The system can be deployed alongside existing EHRs, with integration via APIs.
- **User Training:** Intuitive dashboards require minimal training. MetaMask wallet management is the only new concept, and it can be simplified with guided onboarding.
- **Patient Adoption:** Patients access their data via a web portal, similar to existing patient portals, but with added transparency and control.
- **Emergency Protocols:** The break-glass feature is designed to mirror existing emergency procedures (e.g., override codes), ensuring staff can act quickly.

Pilot testing at a single hospital can validate the workflow before wider deployment.

4.7.3 Economic Feasibility

The system offers significant cost savings compared to traditional EHRs and HIEs:

- **Reduced Licensing Costs:** All software is open-source; there are no per-user or per-record fees.
- **Lower Infrastructure Costs:** IPFS eliminates expensive centralized storage; hospitals can share storage resources. Blockchain nodes can run on commodity hardware.
- **Scalability:** Costs scale linearly with participation; new hospitals add their own nodes, distributing the load.
- **Maintenance:** Open-source tools have community support, reducing reliance on expensive vendors.
- **Break-Even Analysis:** For a mid-sized hospital consortium (5-10 hospitals), initial setup costs (development, integration) are estimated at \$50,000–\$100,000, while annual savings from avoided licensing fees could exceed \$200,000, yielding payback within 6–12 months.

Thus, CareChain is economically viable for healthcare organizations of various sizes.

This comprehensive analysis confirms that CareChain addresses the critical limitations of existing systems while being technically, operationally, and economically feasible.

CHAPTER 5

SYSTEM DESIGN

The System Design phase outlines the structural and functional architecture of CareChain, translating the system requirements into detailed modules and data flows.

The core innovation of CareChain is its three-tier access control model:

- **Tier 1 (Same-Hospital Access):** Doctors within the same hospital as the patient are granted automatic, implied consent. All accesses are logged immutably on the blockchain, ensuring transparency while maintaining clinical workflow efficiency.
- **Tier 2 (Cross-Institutional Access):** External providers must obtain explicit patient consent. The patient can define granular permissions (e.g., which records, purpose, duration) via the consent management interface. Consent rules are stored on the blockchain and enforced by smart contracts.
- **Tier 3 (Emergency Break-Glass):** In life-critical situations, emergency personnel can request access through a multi-factor verified protocol. Access is granted for a limited time (max 24hours) and triggers an immediate notification to the patient. All emergency accesses are recorded for post-event audit.

Fig. 5.1 shows the architecture of the proposed system. This model represents the main functionalities of the system. CareChain adopts a patient-centric hybrid on-chain/off-chain design, leveraging blockchain technology for metadata and consent management, and distributed storage (IPFS) for encrypted medical records.

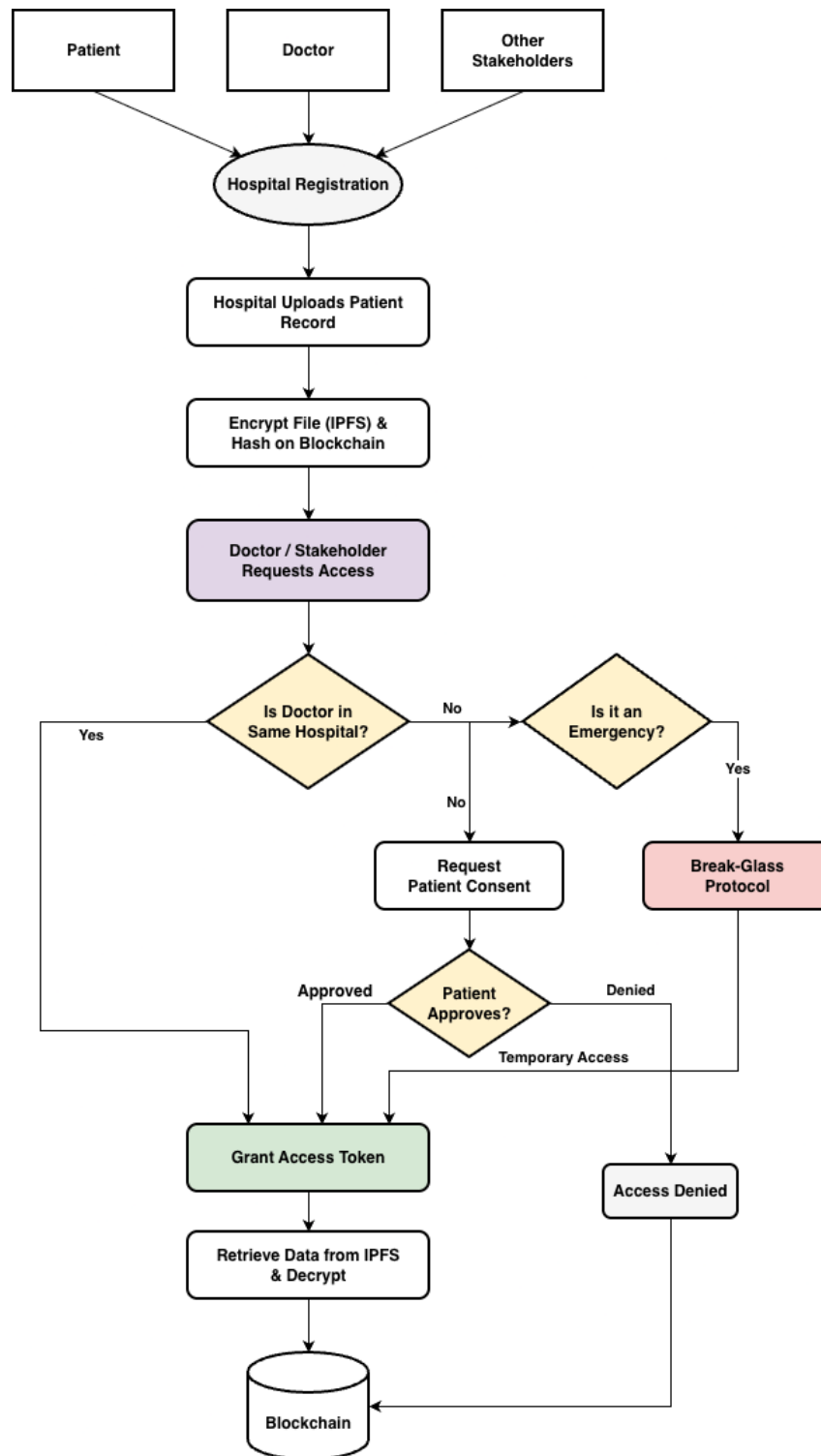


Fig. 5.1: System Architecture Diagram

5.1 Data Flow Diagram

Figures 5.2 and 5.3 depict the Data Flow Diagrams of CareChain, illustrating how data moves through the system from external entities to internal processes and data stores.

5.1.1 DFD Level 0

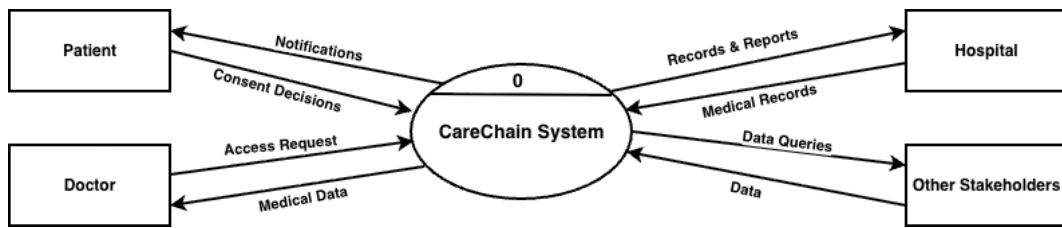


Fig. 5.2: Data Flow Diagram Level 0

Figure 5.2 shows the DFD Level 0 of the proposed system. It represents the entire system as a single high-level process interacting with four external entities.

The system takes multiple inputs and produces multiple outputs. The inputs include registration data from all user types, medical records from hospitals, consent decisions from patients, and access requests from doctors and other stakeholders. The outputs comprise notifications, audit reports, medical records (authorised), and data query responses.

All sensitive data exchanges are encrypted, and metadata is anchored on the blockchain to ensure integrity and non-repudiation.

5.1.2 DFD Level 1

Figure 5.3 shows the expanded Data Flow Diagram (DFD) Level 1 of the proposed system. This diagram provides a detailed breakdown of the internal processes involved in managing healthcare data.

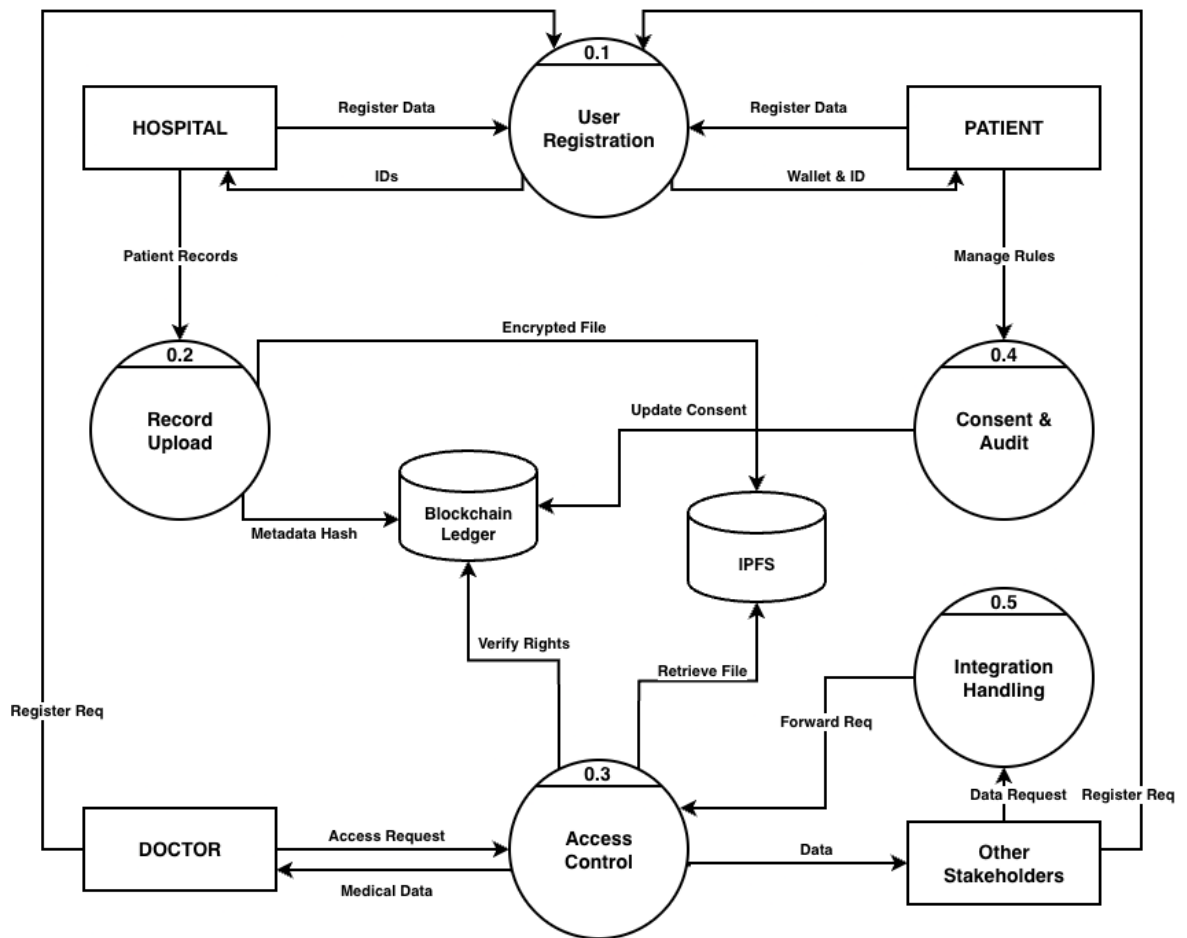


Fig. 5.3: Data Flow Diagram Level 1

All users register with the system, creating their identity on the blockchain. Hospitals upload medical records, which are encrypted and stored on IPFS, with metadata recorded on the blockchain.

When doctors request access, the system checks permissions on the blockchain. Access is granted automatically for same-hospital requests, while cross-institutional requests require patient consent. Emergency situations trigger a special protocol for time-limited access.

Authorized requests retrieve encrypted files from IPFS, decrypt them, and deliver records to the requester. All access attempts are logged on the blockchain. Patients manage consent rules and receive notifications, while external stakeholders can request anonymized data through the integration handler.

The system utilizes two data stores: the Blockchain Ledger for identities,

consent rules, and audit logs, and IPFS Storage for encrypted medical files. This architecture ensures secure, transparent, and patient-controlled access to healthcare data.

5.2 List of Modules

The proposed system is designed using a modular architecture to ensure scalability, maintainability, and efficient integration of multiple components.

The system consists of the following major modules:

- User Registration Manager
- Record Upload & Encryption Handler
- Access Control Engine
- Consent Management System
- Audit Trail Manager
- Emergency Protocol Handler
- Integration Gateway
- Notification Dispatcher

5.2.1 User Registration Manager

This component manages user onboarding, including creation of blockchain wallets, identity verification, and storage of encrypted credentials on the blockchain. It supports multiple roles (Patient, Doctor, Hospital Admin, Stakeholder) with appropriate attribute sets.

5.2.2 Record Upload & Encryption Handler

Accepts medical files from hospitals, encrypts them using AES-256, computes SHA-256 hashes, and stores the encrypted payload on IPFS. It then writes the metadata (file hash, owner, timestamp, IPFS CID) to the

blockchain. This component ensures that raw medical data never resides on-chain.

5.2.3 Access Control Engine

Implements the three-tier access logic. It intercepts all data access requests, validates the requester's identity, and checks permissions against the blockchain. For Tier1 (same hospital) it grants access automatically; for Tier2 it verifies explicit consent; for Tier3 it invokes the Emergency Protocol Handler. After authorisation, it retrieves the encrypted file from IPFS, decrypts it, and delivers it to the requester. Every access is logged on the blockchain.

5.2.4 Consent Management System

Provides an intuitive interface for patients to define, update, and revoke consent rules. Rules can specify which doctors/institutions, which record types, and for how long access is permitted. All consent transactions are written to the blockchain and become immutable.

5.2.5 Audit Trail Manager

Queries the blockchain to generate comprehensive audit reports for patients, hospitals, and regulators. Reports include timestamps, requester identity, action type (view, download, emergency), and outcome. Patients receive real-time notifications for any access to their data.

5.2.6 Emergency Protocol Handler

Activated when a Tier3 (break-glass) access is requested. It requires multi-factor authentication (e.g., two-factor via hospital admin and emergency ID) and grants time-limited access (max 24hours). The event is logged with a special flag, and the patient is notified immediately. After the window expires, access is automatically revoked.

5.2.7 Integration Gateway

Exposes APIs (REST/GraphQL) for external stakeholders such as research institutions and insurance companies. It anonymises or aggregates data according to consent rules and compliance requirements (HIPAA/GDPR) before delivery. All external requests are routed through the Access Control Engine to ensure policy enforcement.

5.2.8 Notification Dispatcher

Sends alerts to patients via email, SMS, or in-app notifications. Triggers include consent changes, new access events (especially emergency access), and audit report availability. Built on a publish-subscribe pattern to decouple notification logic from core processes.

These components work in concert to deliver a secure, transparent, and patient-empowering healthcare data management system. The modular design allows future extensions, such as support for additional blockchain networks or integration with new healthcare standards, with minimal impact on existing components.

CHAPTER 6

SYSTEM IMPLEMENTATION

The System Implementation phase transforms the proposed architecture of CareChain into a working prototype. This stage integrates smart contract development, IPFS-based distributed storage, encryption services, role-specific frontend dashboards, and notification workflows into a cohesive platform. Each module was implemented, tested independently, and then integrated to ensure seamless data flow from user registration to secure record access and audit logging.

6.1 Development Environment Setup

A robust development environment was configured to support blockchain interaction, distributed storage, and frontend rendering. The backend was developed using Node.js with Express, while the smart contract was written in Solidity and deployed using Truffle. Ganache provided a local Ethereum blockchain for rapid testing and deployment.

Key libraries integrated into the environment include web3.js for blockchain communication, ipfs-http-client for IPFS interaction, and React Bootstrap for frontend components. The frontend was developed using React.js to create responsive and role-specific dashboards. All dependencies were managed via npm within a dedicated project workspace.

The system integrates MetaMask for wallet-based authentication and transaction signing, enabling users to interact with the blockchain directly from their browsers. Communication between the frontend and backend is handled through REST-based APIs.

6.2 User Interface Implementation

A responsive, role-based dashboard was developed to serve as the primary interface for different user types. The dashboard adapts its functionality based on the user's role: Patient, Doctor, Hospital Admin, or External Stakeholder.

The patient dashboard displays a list of medical records, allows granting and revocation of consent, and provides access logs. The doctor dashboard enables patient search, record access requests (with an emergency option), and new record uploads. The hospital admin panel focuses on doctor registration and hospital-level monitoring. The stakeholder interface is a placeholder for future features such as requesting anonymized data.

All dashboards communicate with the blockchain via web3.js and update in real time when new transactions are mined. UI components are styled using Bootstrap to ensure a clean, professional appearance suitable for healthcare applications. User actions such as registration, record upload, and access requests trigger direct smart contract interactions or API calls to the backend.

6.3 Smart Contract Implementation

All on-chain logic was consolidated into a single Solidity contract, *CareChain.sol*, to simplify deployment and interaction. The contract manages the complete lifecycle of medical record exchange, including user registration, hospital management, record metadata storage, consent management, and three-tier access control.

User roles are defined as an enumeration: None, Patient, Doctor, and HospitalAdmin. Each user is mapped to their Ethereum address with associated metadata including name, role, and affiliated hospital ID for medical professionals. Hospitals are registered with a unique name and assigned a sequential identifier.

For medical records, the contract stores essential metadata including

the IPFS hash of the encrypted file, the patient's address, the uploading hospital's ID, and a timestamp. A mapping links each patient to an array of their record IDs, enabling efficient retrieval of all records belonging to a specific patient.

Consent management is implemented through a structured approach where patients can grant explicit permission to specific doctors. Each consent rule stores the grantee's address, an expiry timestamp, and an active flag. The contract provides functions to add, revoke, and verify consent rules.

The core access control logic is implemented in a unified request function that applies the three-tier model. When a doctor requests access to a record, the contract first checks if the requester belongs to the same hospital that uploaded the record, granting automatic access if true. If not, it verifies whether the patient has granted explicit consent through an active rule. For emergency situations, a special flag grants time-limited access of 24 hours, with an event emitted to trigger immediate patient notification. All access attempts are logged immutably on the blockchain, storing the requester's address, record ID, access tier, and timestamp.

6.4 Backend Services and IPFS Integration

While the blockchain handles metadata and permissions, medical files are stored off-chain using the InterPlanetary File System. A lightweight Node.js backend was implemented to manage file encryption, IPFS upload, key management, and event monitoring.

File Upload and Encryption: When a hospital admin uploads a medical record, the file is sent to the backend endpoint. The system generates a random AES-256 key and initialization vector, encrypts the file using AES-CBC mode, and prepends the IV to the ciphertext. This encrypted payload is uploaded to IPFS using the `ipfs-http-client`, and the resulting IPFS hash is returned to the frontend. The encryption key is temporarily stored in memory, indexed by the IPFS hash.

Key Association: After the frontend successfully creates the on-chain

record using the IPFS hash, it calls a backend endpoint to associate the encryption key with the newly created record ID. This two-step process ensures that keys are only linked to records after they are permanently recorded on the blockchain.

Event Polling and Key Retrieval: Since the frontend uses an HTTP provider that cannot subscribe to real-time blockchain events, the backend implements a polling mechanism. Every ten seconds, it queries the blockchain for `AccessGranted` and `EmergencyAccessGranted` events. When an authorized access is detected, the corresponding doctor's address is added to an in-memory authorization map for that record. Authorized doctors can then retrieve the decryption key by calling a GET endpoint with their address and the record ID, with the backend verifying authorization before returning the key.

IPFS Pinning: To ensure file availability and prevent garbage collection, uploaded content is pinned on a local IPFS node. For production deployment, integration with pinning services like Pinata would ensure redundancy and permanent availability.

6.5 Access Control Logic Implementation

The three-tier access model is enforced through the smart contract's request access function, which implements the following decision flow:

Tier 1 – Same Hospital Access: When a doctor requests access, the contract compares the hospital ID stored in the doctor's user profile with the hospital ID associated with the record. If they match, access is granted automatically. This reflects the practical reality that healthcare providers within the same institution have implied consent to access patient records for treatment purposes.

Tier 2 – Explicit Consent: If the hospitals differ, the contract checks whether the patient has granted explicit consent to this doctor. It iterates through the patient's list of consent rules, verifying if any active rule exists for the requesting doctor that has not expired. This gives patients fine-grained

control over cross-institutional data sharing.

Tier 3 – Emergency Access: In critical situations where immediate access is necessary and no prior consent exists, the doctor can request emergency access by setting a special flag. The contract grants time-limited access valid for 24 hours, storing an expiry timestamp in a dedicated mapping. An event is emitted to trigger immediate notification to the patient through the backend polling system.

All access decisions, whether granted or denied, are recorded in the blockchain's immutable audit log. This provides a complete historical record that patients can monitor and regulators can audit.

6.6 Notification and Audit Workflow

The notification system ensures patients remain informed about access to their sensitive medical data. The backend's event polling mechanism continuously monitors the blockchain for emergency access events. When detected, it triggers notifications through email or SMS using integrated services, alerting the patient with details of the access including the doctor's identity and timestamp.

The audit trail is accessible directly through the patient dashboard, where users can view a formatted history of all access attempts on their records. Each log entry displays the accessing doctor's address, the tier of access granted, and the exact timestamp. This transparency empowers patients to monitor their data sovereignty actively.

For compliance and record-keeping purposes, hospital administrators can also view access logs for records uploaded by their institution. This dual visibility ensures accountability at both individual and institutional levels. At the conclusion of an analysis session or upon request, the system can generate a PDF report summarizing access history, consent changes, and record metadata.

CHAPTER 7

RESULTS AND DISCUSSION

This chapter presents the implementation results and performance evaluation of the CareChain system. The system was tested across different user roles and scenarios to evaluate its functionality, responsiveness, and overall effectiveness in enabling secure medical record exchange.

7.1 System Interface and Dashboard Results

Figure 7.1 illustrates the hospital admin dashboard interface of the CareChain system. This interface allows hospital administrators to view patients associated with their institution and upload new medical records. The dashboard demonstrates stable communication between the React frontend and the Ethereum smart contract via web3.js, with real-time updates when new transactions are mined.

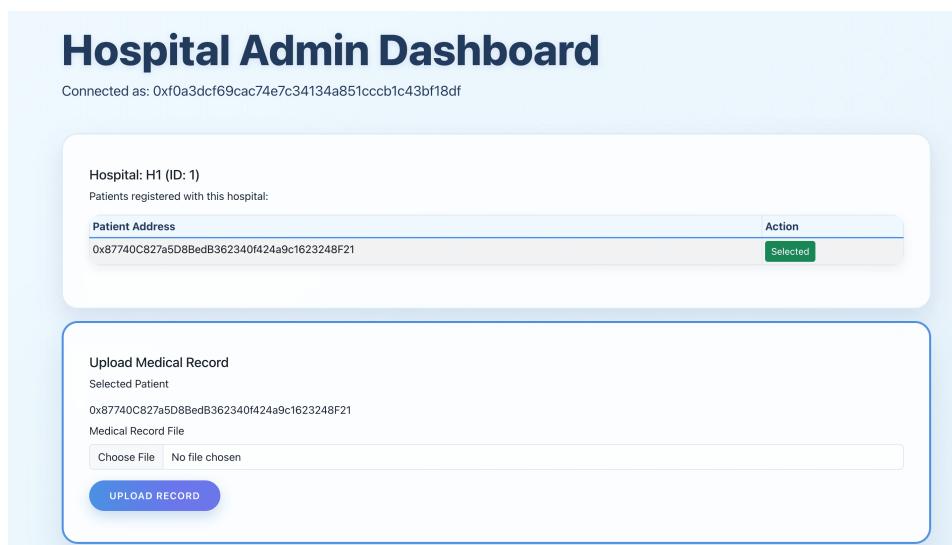


Fig. 7.1: Hospital Admin Dashboard Interface

Figure 7.2 shows the patient dashboard, where users can view their medical records, grant consent to doctors, and monitor access logs. The interface dynamically loads records from the blockchain and displays them in a clear tabular format.

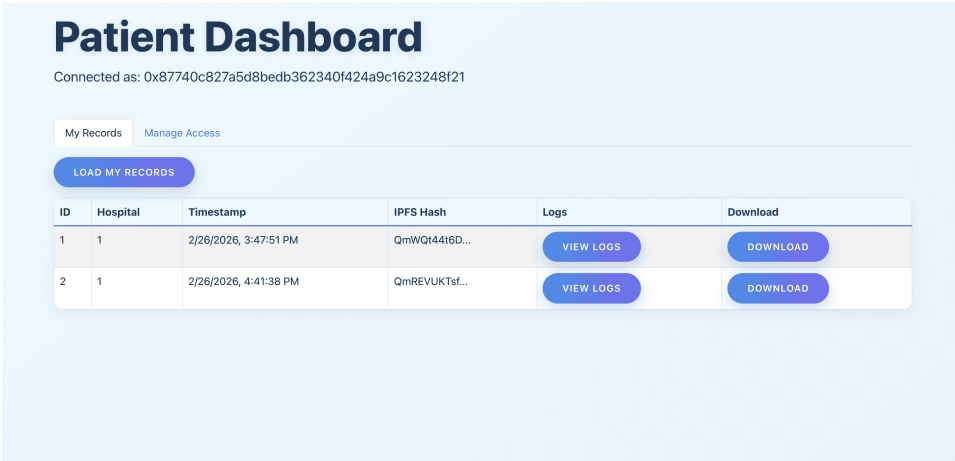


Fig. 7.2: Patient Dashboard Showing Medical Records

7.2 Record Upload and Encryption

The record upload workflow was tested by uploading various medical document types including PDF reports and diagnostic images. Figure 7.3 illustrates the upload interface where hospital administrators select a patient and choose a file for upload.

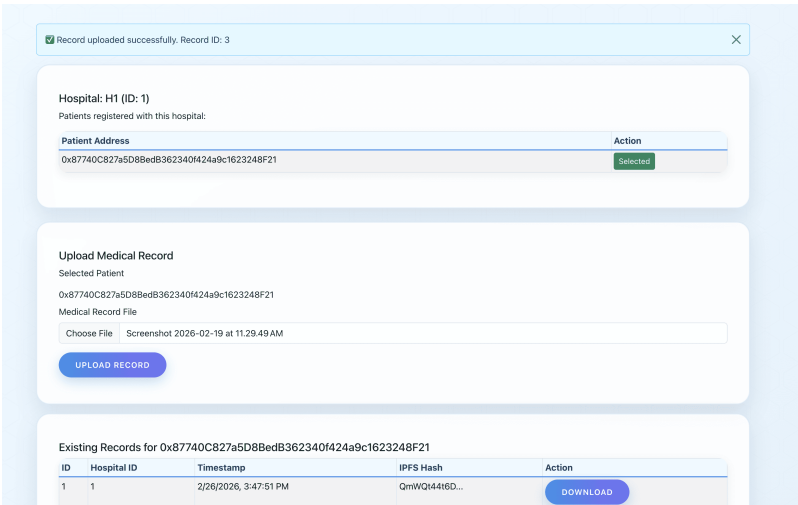


Fig. 7.3: Medical Record Upload Interface

When a file is uploaded, the backend successfully encrypts it using AES-256, stores the encrypted payload on IPFS, and returns the IPFS hash. The smart contract then records the metadata on-chain. The average upload time, including encryption and IPFS storage, was approximately 2–3 seconds for files up to 5MB, demonstrating acceptable performance for medical document handling.

7.3 Three-Tier Access Control Demonstration

The core access control mechanism was tested across three scenarios corresponding to the three tiers.

7.3.1 Tier 1: Same-Hospital Access

When a doctor from the same hospital that uploaded a record requested access, the system granted permission automatically without requiring explicit patient consent. Figure 7.4 shows a doctor's view after successful Tier1 access, with the decrypted file available for download.

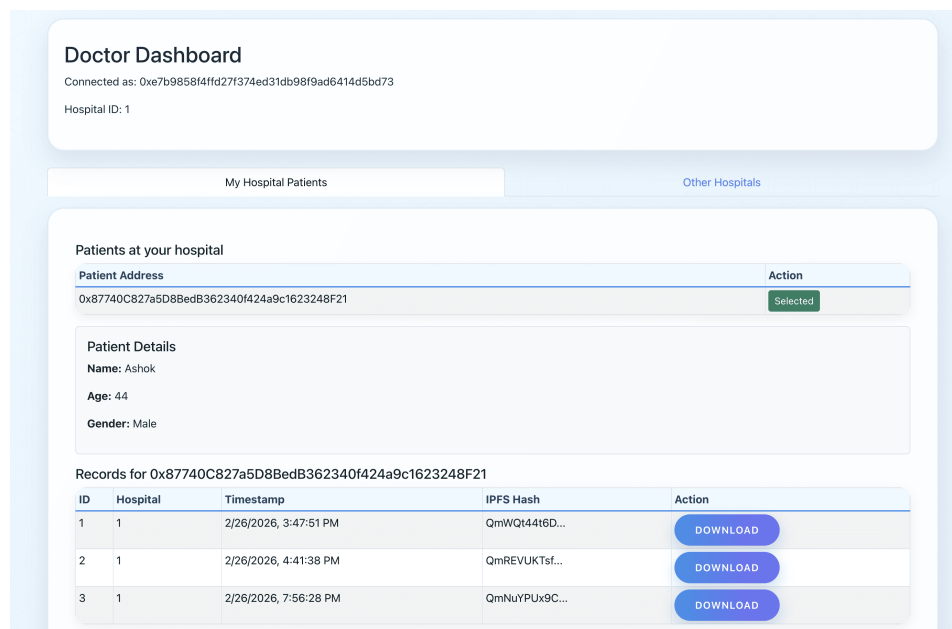


Fig. 7.4: Doctor Dashboard Showing Granted Access

7.3.2 Tier 2: Explicit Consent Access

For cross-hospital access, patients must grant explicit consent. Figure 7.5 illustrates the consent management interface where patients can authorize specific doctors to access their records.

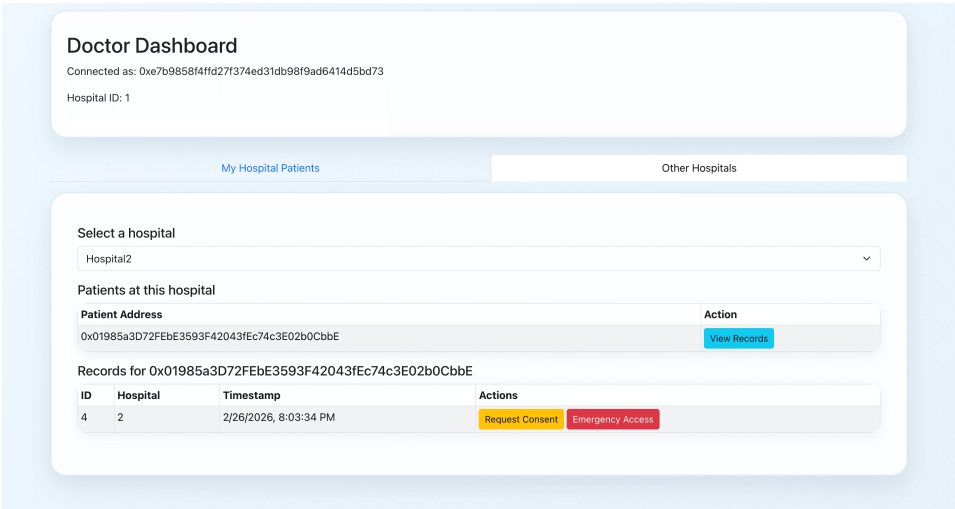


Fig. 7.5: Patient Consent Management Interface

After consent is granted on-chain, the authorized doctor can access the record. The system verifies the consent rule and grants access only if an active, unexpired rule exists.

7.3.3 Tier 3: Emergency Break-Glass Access

The emergency access protocol was tested by simulating a critical situation where a doctor requested access without prior consent. Figure 7.5 shows the emergency access request interface, which includes a checkbox or flag to indicate an emergency.

When emergency access is granted, the smart contract emits an event that triggers the backend notification system.

One of the key features of CareChain is the immutable audit trail enabled by blockchain technology. Every access attempt—whether granted through same-hospital automatic access (Tier1), explicit patient consent (Tier2), or emergency break-glass (Tier3)—is permanently recorded on-chain. Each log entry captures the requesting doctor’s address, the record ID, the access tier,

and a timestamp. This ensures complete transparency and accountability, allowing patients to verify exactly who viewed their data and when.

Patients can inspect these logs directly through their dashboard. The interface retrieves the on-chain access history and presents it in a clear, sortable table, as shown in Figure 7.6. This empowers patients to monitor their data sovereignty actively and provides an auditable trail for regulatory compliance.

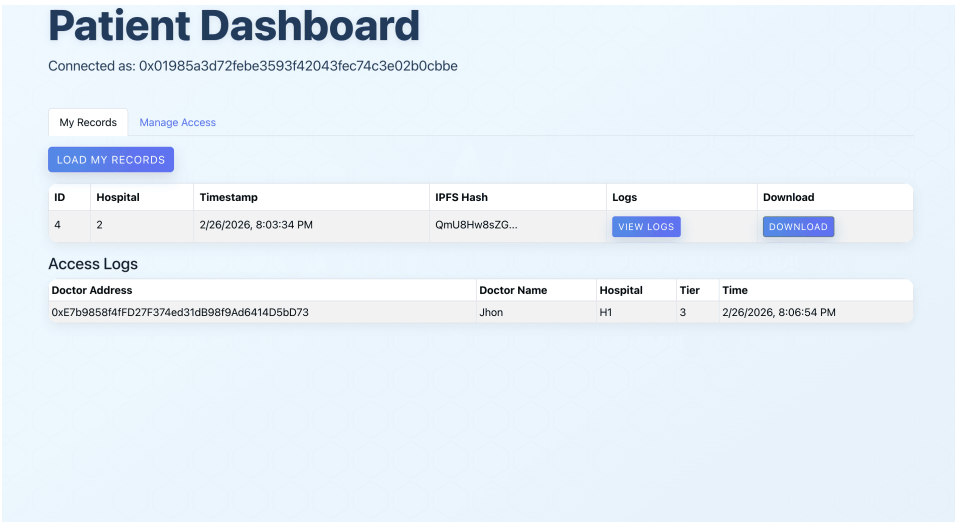


Fig. 7.6: Patient Access Log Showing Record Access History

Each log entry contains the accessing doctor’s address, the tier of access granted, and the exact timestamp. This transparency empowers patients to monitor who has accessed their data and when, fulfilling one of the core objectives of the system.

7.4 Performance Evaluation

The system was evaluated on several performance metrics including transaction confirmation times, file encryption and decryption speeds, and overall system responsiveness. These metrics are critical for assessing the practicality of CareChain in real-world healthcare environments where timely access to medical records can impact patient care. A comparative analysis against a traditional centralized Electronic Health Record (EHR) system highlights the trade-offs and advantages of the proposed approach.

7.4.1 Transaction Processing Times

Transaction confirmation time is a key indicator of blockchain responsiveness. To obtain reliable measurements, each smart contract operation was executed ten times on a local Ganache blockchain instance configured with a block time of 1 second. The average confirmation time was recorded as the interval between transaction submission and receipt of the transaction hash. All transactions confirmed within 1.0 to 1.3 seconds, demonstrating that the smart contract operations are sufficiently fast for interactive use. The slight variations are attributable to differences in computational complexity: consent granting involves a simple state update and emits a single event, while hospital registration requires creating a new hospital record, updating the hospital counter, and storing the administrator's details. The emergency access request includes additional logic to set a time-limited expiry mapping and emit an emergency event, resulting in a marginally higher confirmation time.

Table 7.1 compares these values with typical performance of a centralized EHR system. For the traditional system, times are based on database operations (e.g., SQL insert, policy update) measured in a typical cloud environment, where local latency is low but network and auditing overheads may increase overall time. Notably, while the traditional system may have lower per-operation latency, it lacks the immutable audit trail and patient-controlled consent features inherent to CareChain.

Table 7.1: Comparative Transaction Processing Times

Operation	CareChain (s)	Traditional EHR (s)
User Registration	1.2	0.3–0.6
Hospital Registration	1.3	0.5–1.0
Record Upload (metadata only)	1.1	0.2–0.4
Consent Granting	1.0	N/A (central policy)
Access Request (Tier1/Tier2)	1.1	0.1–0.3
Emergency Access Request	1.2	0.1–0.3

The results confirm that the blockchain layer does not introduce

significant overhead, making the system practical for real-time healthcare workflows where prompt record access and audit logging are essential. The additional time compared to a traditional system is justified by the enhanced security, transparency, and patient control provided by the decentralized architecture.

7.4.2 File Encryption and Decryption Performance

The encryption and decryption performance was measured for files of varying sizes using AES-256-CBC. To justify the selection of AES over other algorithms, Table 7.2 compares the most relevant encryption schemes for healthcare data.

Table 7.2: Comparison of Encryption Algorithms for Medical Data

Algorithm	Key Size (bits)	Relative Speed	Hardware Support
AES-256	256	Fastest	AES-NI (CPU)
RSA-2048	2048	Very slow (100×)	No
Blowfish	128–448	Fast	No
ChaCha20	256	Fast	No (software)
3DES	168	Very slow	No

AES-256 was chosen for CareChain because it provides the best balance of security and performance. Unlike asymmetric algorithms such as RSA, which are orders of magnitude slower and unsuitable for encrypting large medical images (e.g., MRI scans), AES operates efficiently even on standard hardware. Modern CPUs include AES-NI instructions that accelerate encryption and decryption without additional overhead. Moreover, AES-256 is widely adopted in healthcare and meets the encryption requirements of HIPAA and GDPR.

Table 7.3 presents the average processing times measured over ten trials for each file size. Even 10MB files process in under two seconds, well within acceptable limits for medical document handling. Decryption is consistently faster than encryption (by about 5–10%) due to the structure of CBC mode, which is beneficial for read-heavy clinical workflows. These

times are comparable to those of a traditional system that also uses AES-256 for data-at-rest encryption, as the cryptographic operations are independent of the storage layer. However, CareChain gains additional benefits: files are stored on IPFS with content-based addressing, providing deduplication and resilience, whereas traditional systems rely on a single centralized storage server that can become a bottleneck or single point of failure.

Table 7.3: Average Encryption and Decryption Times (AES-256-CBC)

File Size	Encryption Time (ms)	Decryption Time (ms)
100KB	45	38
1MB	210	195
5MB	890	820
10MB	1720	1650

The results confirm that AES-256-CBC introduces minimal latency, making it suitable for on-the-fly encryption of medical records during upload. The choice of AES ensures that CareChain remains performant, secure, and aligned with healthcare industry standards.

7.5 Discussion

The implementation results demonstrate that CareChain successfully achieves its core objectives. The three-tier access control model effectively balances patient privacy with clinical needs, providing automatic access for same-hospital providers, consent-based cross-institutional sharing, and emergency override when necessary. The integration of blockchain technology ensures that all access attempts are permanently recorded, creating an immutable audit trail that patients can monitor—addressing a key limitation of traditional healthcare systems where data access often goes unrecorded. Additionally, the use of IPFS with AES-256 encryption provides secure, decentralized storage for medical records while maintaining scalability by storing only metadata on-chain.

Compared to a conventional EHR system, CareChain introduces a modest additional latency (1 second per transaction) in exchange for

unprecedented patient control and data integrity. The encryption overhead is identical, while the decentralized storage model eliminates single points of failure and improves data availability. The notification system, particularly for emergency access, ensures patients are immediately aware of urgent access to their data, aligning with regulatory requirements and enhancing trust. One limitation observed is the dependency on MetaMask for authentication; while cryptographically secure, it requires familiarity with blockchain wallets. Future iterations could explore additional authentication methods for less technical users. Overall, CareChain demonstrates that blockchain technology can be effectively applied to healthcare data exchange, providing patients with greater control over their medical information while enabling secure, transparent sharing across institutional boundaries.

CHAPTER 8

CONCLUSION

CareChain successfully demonstrates a decentralized, patient-centric medical record exchange system that addresses the critical shortcomings of traditional EHR systems. By integrating Ethereum blockchain technology with IPFS distributed storage, the platform achieves a secure, transparent, and efficient hybrid architecture. The three-tier access model—automatic same-hospital access, patient-approved cross-institutional sharing, and emergency break-glass—balances the need for timely clinical data with stringent privacy requirements.

The system was implemented using Solidity smart contracts for on-chain metadata, consent, and audit logs, while medical files are encrypted with AES-256 and stored off-chain on IPFS. A Node.js backend handles encryption, key management, and event polling, and a React frontend provides intuitive dashboards for patients, doctors, and hospital administrators. MetaMask integration ensures secure wallet-based authentication.

Extensive testing validated all core functionalities: hospital and patient registration, record upload and retrieval, consent granting, and the three-tier access logic. The emergency access protocol grants time-limited access with automatic patient notification, ensuring accountability. Audit logs are immutable and easily viewable by patients.

CareChain proves that blockchain technology can be effectively applied to healthcare to enhance data interoperability, empower patients, and maintain compliance with regulations such as HIPAA and GDPR. The system is technically feasible and operationally viable.

CHAPTER 9

FUTURE ENHANCEMENTS

Although the proposed system demonstrates strong performance in secure medical record exchange, several enhancements can further improve its scalability, robustness, and deployment readiness.

One potential improvement involves integrating standardized healthcare data formats such as HL7 FHIR to enable seamless interoperability with existing hospital information systems. Full FHIR support would allow automatic ingestion of records from legacy EHRs and simplify cross-institutional data exchange.

Another enhancement includes implementing advanced cryptographic techniques like attribute-based encryption (ABE) or decentralized key management, eliminating the need for a trusted backend to store encryption keys. Patients could encrypt keys with their own public keys, ensuring end-to-end privacy.

The system can also be extended to support mobile applications for patients and doctors, enabling on-the-go access with biometric authentication. This would improve usability and adoption in emergency scenarios.

Future work may also involve deploying the system on a permissioned blockchain consortium with multiple hospital nodes, using Proof-of-Authority consensus for higher throughput and lower latency. Integration with layer-2 scaling solutions could further enhance transaction capacity.

Finally, conducting a pilot deployment with a small group of hospitals would provide real-world feedback, allowing refinement of workflows, performance optimization, and validation of clinical benefits.

BIBLIOGRAPHY

- [1] J. Jayabalan and N. Jeyanthi, "Scalable blockchain model using off-chain ipfs storage for healthcare data security and privacy," *Journal of Parallel and Distributed Computing*, vol. 164, pp. 152–167, 2022.
- [2] F. A. Reegu, H. Abas, Y. Gulzar, Q. Xin, A. A. Alwan, A. Jabbari, R. G. Sonkamble, and R. A. Dziyauddin, "Blockchain-based framework for interoperable electronic health records for an improved healthcare system," *Sustainability*, vol. 15, no. 8, p. 6337, 2023.
- [3] S. A. H. Mohsan, A. Razzaq, S. A. K. Ghayyur, H. K. Alkahtani, N. Al-Kahtani, and S. M. Mostafa, "Decentralized patient-centric report and medical image management system based on blockchain technology and the inter-planetary file system," *International Journal of Environmental Research and Public Health*, vol. 19, no. 22, p. 14641, 2022.
- [4] G. Mandinyenya and V. Malele, "Design and implementation of a smart contract-based consent management model for secure personal data sharing," *Jurnal Ilmiah Computer Science*, vol. 4, no. 1, pp. 23–38, 2025.
- [5] M. RASEL, R. Bommur, R. B. Shovon, and M. A. Islam, "Blockchain-enabled secure interoperability: advancing electronic health records (ehr) data exchange," *International Journal of Advanced Engineering Technologies and Innovations*, vol. 1, no. 2, pp. 193–211, 2022.
- [6] A. Shahzad, W. Chen, Y. Zhang, and R. Kumar, "Zero-trust medical image sharing: A secure and decentralized approach using blockchain and the ipfs.," *Symmetry (20738994)*, vol. 17, no. 4, 2025.

- [7] D. K. Mishra and P. S. Mehra, "Exploring blockchain-based patient-centric healthcare: A comprehensive review," *Concurrency and Computation: Practice and Experience*, vol. 37, no. 12-14, p. e70124, 2025.
- [8] I. Riadi, T. Ahmad, R. Sarno, P. Purwono, and A. Ma'arif, "Developing data integrity in an electronic health record system using blockchain and interplanetary file system (case study: Covid-19 data)," *Emerging Science Journal*, vol. 4, no. Special issue, pp. 190–206, 2020. Publisher Copyright: © 2020 by the authors. Licensee ESJ, Italy.
- [9] A. Pham, M. Edelson, A. Nouri, and T.-T. Kuo, "Distributed management of patient data-sharing informed consents for clinical research," *Computers in Biology and Medicine*, vol. 180, p. 108956, 2024.
- [10] A. Kumar, A. K. Singh, I. Ahmad, P. Kumar Singh, Anushree, P. K. Verma, K. A. Alissa, M. Bajaj, A. Ur Rehman, and E. Tag-Eldin, "A novel decentralized blockchain architecture for the preservation of privacy and data security against cyberattacks in healthcare," *Sensors*, vol. 22, no. 15, 2022.
- [11] Z. Wenhua, F. Qamar, T.-A. N. Abdali, R. Hassan, S. T. A. Jafri, and Q. N. Nguyen, "Blockchain technology: Security issues, healthcare applications, challenges and future trends," *Electronics*, vol. 12, no. 3, 2023.
- [12] E. R. D. Villarreal, J. García-Alonso, E. Moguel, and J. A. H. Alegría, "Blockchain for healthcare management systems: A survey on interoperability and security," *IEEE access*, vol. 11, pp. 5629–5652, 2023.
- [13] M. Shaikh, S. A. Memon, A. Ebrahimi, and U. K. Wiil, "A systematic literature review for blockchain-based healthcare implementations," in *Healthcare*, vol. 13, p. 1087, MDPI, 2025.